Python for Artificial Intelligence

Neural Networks in PyTorch

Week 4, Day 5

Today's Project: RideNet

- Yesterday you hand-scored one ride with weights you picked. Today the network learns its own weights, for real
- Hour 1: prepare the ride data, then build RideNet the hard way: plain NumPy, no library, every formula by hand
- Hour 2: build the same network the easier way in TensorFlow (Keras), then start on PyTorch: tensors and defining the network
- Hour 3: PyTorch's training loop, then compare all three builds with each other and with Day 18's classical models

Hour 1

Data and the Hard Way: No Library

PROJECT

Preparing ride_requests.csv for a Neural Network

- Networks work with numbers in arrays, not DataFrames with text columns, so the data needs preparing first
- Features (distance_km, time_of_day, area_type) are one-hot encoded into the same 6 columns as Day 18, then scaled to similar ranges, networks train better when inputs aren't wildly different sizes
- Same 80/20 train/test split as Day 18, with a fifth of the training rides held back as validation data to monitor during training, so RideNet is scored on the same 200 test rides as the classical models
- This is the same ride_requests.csv from Day 17-18, now predicted with a neural network instead of Logistic Regression or a Decision Tree

EXAMPLE

Scaling the Ride Data

StandardScaler scales the features (fit on the training data only). The result is plain NumPy arrays, used by all three builds today.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# same 80/20 test split as Day 18, then 20% of training kept for validation
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2,
random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val, X_test = scaler.transform(X_val), scaler.transform(X_test)
y_train, y_val, y_test = [v.values.reshape(-1, 1) for v in (y_train, y_val, y_test)]
```

CONCEPT

Three Ways to Build a Network

The same RideNet can be built with no library at all, with TensorFlow (Keras), or with PyTorch. Today we go from the hardest way to the easier ones.

	No library (NumPy)	TensorFlow (Keras)	PyTorch
Order today	1. The hard way	2. Easier	3. Easiest to see inside
Gradients	You derive and code every one by hand	Automatic (GradientTape)	Automatic (autograd)
GPU	No	Yes	Yes
Layers and optimizers	You write them	Dense, built-in optimizers	nn.Linear, torch.optim
Training loop	Written by hand	One call: model.fit()	Written by hand (4 steps)
Best for	Learning exactly what happens	Quick builds and deployment	Research and flexible experiments

EXAMPLE

No Library: Setup and Forward Pass

RideNet in plain NumPy. X_train holds the 6 scaled ride columns and y_train the 0/1 labels, prepared on the previous slides.

```
import numpy as np

rng = np.random.default_rng(42)
W1 = rng.normal(0, 0.5, (6, 8));  b1 = np.zeros(8)    # hidden layer
W2 = rng.normal(0, 0.5, (8, 1));  b2 = np.zeros(1)    # output layer
sigmoid = lambda z: 1 / (1 + np.exp(-z))

# forward pass
z1 = X_train @ W1 + b1
a1 = np.maximum(0, z1)                  # ReLU
p  = sigmoid(a1 @ W2 + b2)              # quick-pickup probabilities

# loss: mean squared error
loss = np.mean((p - y_train) ** 2)
```

EXAMPLE

No Library: Backward Pass and Update

Backpropagation written out by hand: the chain rule, layer by layer, then one gradient descent step.

```
# backward pass (chain rule)
dz2 = 2 * (p - y_train) * p * (1 - p) / len(y_train)   # error at the output (MSE, then sigmoid)
dW2 = a1.T @ dz2;  db2 = dz2.sum(axis=0)
dz1 = (dz2 @ W2.T) * (z1 > 0)            # ReLU passes gradient only where z1 > 0
dW1 = X_train.T @ dz1;  db1 = dz1.sum(axis=0)

# update: gradient descent
lr = 0.5
W1 -= lr * dW1;  b1 -= lr * db1
W2 -= lr * dW2;  b2 -= lr * db2

# put forward pass + loss + backward pass + update in a loop of 500 epochs
```

CONCEPT

What a Framework Does for You

Compare the by-hand version with what a framework (Keras or PyTorch) does for the same network.

Job	By hand (NumPy)	With a framework (Keras or PyTorch)
Weights and biases	Create W1, b1, W2, b2 yourself	Dense / nn.Linear creates and tracks them
Gradients	Write every formula (dz2, dW2, dz1, dW1…)	Automatic: GradientTape or loss.backward()
Weight updates	W -= lr * dW for every parameter	The optimizer does it for you
Changing the network	Re-derive the gradients	Change the layer definition
Risk of mistakes	High: one wrong formula breaks training silently	Low: autograd is thoroughly tested

Hour 2

The Easier Way: TensorFlow, then PyTorch Basics

EXAMPLE

RideNet in TensorFlow (Keras)

The same 6-8-1 network, now with Keras. It hides the loop and the calculus inside compile() and fit().

```
import tensorflow as tf

tf.keras.utils.set_random_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(6,)),
    tf.keras.layers.Dense(8, activation="relu"),       # hidden layer
    tf.keras.layers.Dense(1, activation="sigmoid"),    # output layer
])
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
              loss="mse", metrics=["accuracy"])

model.fit(X_train, y_train, validation_data=(X_val, y_val),
          epochs=100, batch_size=len(X_train))
loss, acc = model.evaluate(X_test, y_test)      # test accuracy: 0.650
```

CONCEPT

Keras: What Each Line Does

Compare with the NumPy version: no gradients, no update lines, no loop.

Line	What it does
Sequential([...])	Stacks layers in order, like the forward pass you wrote by hand
Dense(8, activation="relu")	Hidden layer: 6 inputs to 8 neurons with ReLU (W1, b1 created for you)
Dense(1, activation="sigmoid")	Output layer: 8 values to one quick-pickup probability
compile(optimizer, loss)	Picks Adam and mean squared error, the loss you coded by hand
fit(...)	Forward pass, loss, gradients and updates for every epoch, in one call
evaluate(X_test, y_test)	Scores the trained network on the 200 unseen test rides

CONCEPT

What Is PyTorch?

- PyTorch is the most widely used framework for building and training neural networks in Python, and it is the one most modern AI research is written in
- It handles two things you don't want to write by hand: computing gradients automatically, and running computations fast (optionally on a GPU)
- Its core data structure is the tensor, which behaves almost exactly like a NumPy array
- If you can use NumPy, you already know most of what you need to start using PyTorch

EXAMPLE

Tensors: NumPy's Cousin

A tensor is created and used almost identically to a NumPy array.

```
import torch

x = torch.tensor([0.5, 0.2, 0.9])    # one ride's scaled features
w = torch.tensor([0.4, -0.3, 0.8])   # weights

z = torch.dot(x, w) + 0.1
print(z)              # tensor(0.9600)
print(z.shape)        # torch.Size([])
```

EXAMPLE

From NumPy Arrays to Tensors

The arrays prepared earlier become float32 tensors, the data type PyTorch networks expect.

```
import torch

to_t = lambda a: torch.tensor(a, dtype=torch.float32)
X_train, X_val, X_test = to_t(X_train), to_t(X_val), to_t(X_test)
y_train, y_val, y_test = to_t(y_train), to_t(y_val), to_t(y_test)

print(X_train.shape)    # torch.Size([640, 6])
print(y_train.shape)    # torch.Size([640, 1])
```

CONCEPT

Why Not Just Use NumPy?

- Tensors can run on a GPU, where the same matrix operations happen far faster, essential for large networks
- Tensors support autograd: PyTorch automatically tracks every operation and can compute gradients for you
- That second point replaces backpropagation's math entirely, you call one method, PyTorch does the calculus
- For today's small network, speed won't matter, but the autograd habit you build today scales to everything later

EXAMPLE

Defining RideNet

nn.Module is the standard way to define a network's layers and how data flows through them.

```
import torch.nn as nn

class RideNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(6, 8)   # 6 ride columns -> 8 hidden neurons
        self.output = nn.Linear(8, 1)   # 8 hidden -> 1 output (quick pickup?)

    def forward(self, x):
        x = torch.relu(self.hidden(x))
        x = torch.sigmoid(self.output(x))
        return x

torch.manual_seed(42)               # same starting weights every run
model = RideNet()
```

CONCEPT

Reading That Network

- nn.Linear(6, 8) is one layer: it takes 6 inputs (distance_km plus the one-hot columns for time_of_day and area_type, as in Day 18) and produces 8 numbers out, the weighted-sum step from Day 19
- torch.relu and torch.sigmoid are the activation functions from yesterday, applied right after each layer
- forward() describes the forward pass: how a ride's features flow through the layers to produce a quick-pickup probability
- This is the exact same anatomy as yesterday's hand-scored neuron, now written as runnable code

CONCEPT

RideNet, Layer by Layer

What model.parameters() counts: every weight and bias the optimizer will adjust.

Layer	Shape	Parameters	What it does
Input	6 columns	0	One ride's 6 scaled features
hidden: nn.Linear(6, 8)	6 in, 8 out	6 x 8 + 8 = 56	Weighted sums for 8 hidden neurons
ReLU	8 values	0	Keeps positives, zeroes out negatives
output: nn.Linear(8, 1)	8 in, 1 out	8 x 1 + 1 = 9	One weighted sum of the hidden values
Sigmoid	1 value	0	Turns that sum into a quick-pickup probability
Total		65	Weights and biases the optimizer learns

QUICK CHECK

Quick Check

In RideNet above, why is the final activation sigmoid rather than ReLU?

ANSWER

Quick Check: Answer

```
# The task is binary classification (Quick Pickup or Long
# Wait), so the output needs to be a probability between 0
# and 1.
#
# Sigmoid squashes any number into that 0-to-1 range, exactly
# like Logistic Regression's output on Day 18. ReLU only
# clips negative numbers to zero, it doesn't bound the top,
# so it isn't suitable as a final layer for a probability.
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

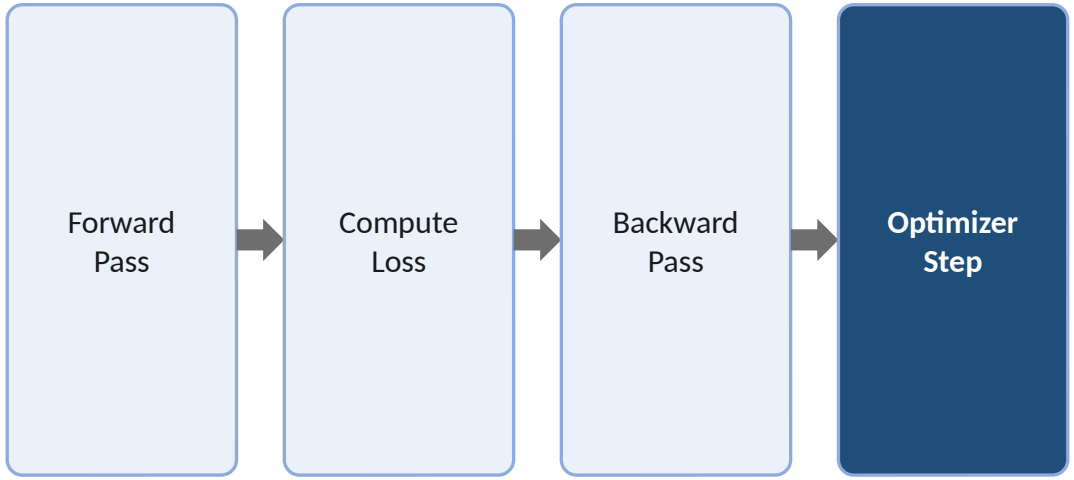
Hour 3

PyTorch Training, Then Comparing All Three

CONCEPT

Four Steps, Repeated Every Batch

- Every PyTorch training loop repeats the same four steps, for every batch, every epoch
- Forward pass: feed ride features through the model to get a quick-pickup probability
- Compute loss: compare that probability to what actually happened with a loss function
- Backward pass & update: compute gradients automatically, then nudge every weight to reduce the loss



EXAMPLE

Loss Function & Optimizer

The loss function matches the task; the optimizer is the algorithm that performs gradient descent.

```
loss_fn = nn.MSELoss()                    # mean squared error, the same loss as Day 17 regression
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
# Adam is a smarter, adaptive version of plain gradient descent from Day 19
```

EXAMPLE

The Training Loop, Written Out

This is the forward/loss/backward/update cycle from the diagram, now as real code.

```
num_epochs = 100

for epoch in range(num_epochs):
    predictions = model(X_train)          # forward pass
    loss = loss_fn(predictions, y_train)  # compute loss

    optimizer.zero_grad()                 # clear old gradients
    loss.backward()                       # backward pass (autograd!)
    optimizer.step()                      # update every weight
```

CONCEPT

The Training Loop, Line by Line

Every line maps to an idea from Day 19.

Code	Day 19 idea	What it does
predictions = model(X_train)	Forward propagation	Pushes every training ride through the network
loss = loss_fn(predictions, y_train)	Loss calculation	Scores how wrong the probabilities are
optimizer.zero_grad()	(housekeeping)	Clears the gradients left from the last epoch
loss.backward()	Backpropagation	Works out each weight's gradient with the chain rule
optimizer.step()	Gradient descent (Adam)	Moves every weight a small step downhill

CONCEPT

loss.backward(): Autograd in Action

- loss.backward() is backpropagation from Day 19, computed automatically for every weight in RideNet
- optimizer.zero_grad() is easy to forget, gradients accumulate by default, so clear them before every new batch
- optimizer.step() is the actual gradient descent update, moving every weight slightly downhill
- These three lines, in this order, appear in almost every PyTorch training script you will ever write

QUICK CHECK

Quick Check

What would likely go wrong if you forgot to call optimizer.zero_grad() before each loss.backward()?

ANSWER

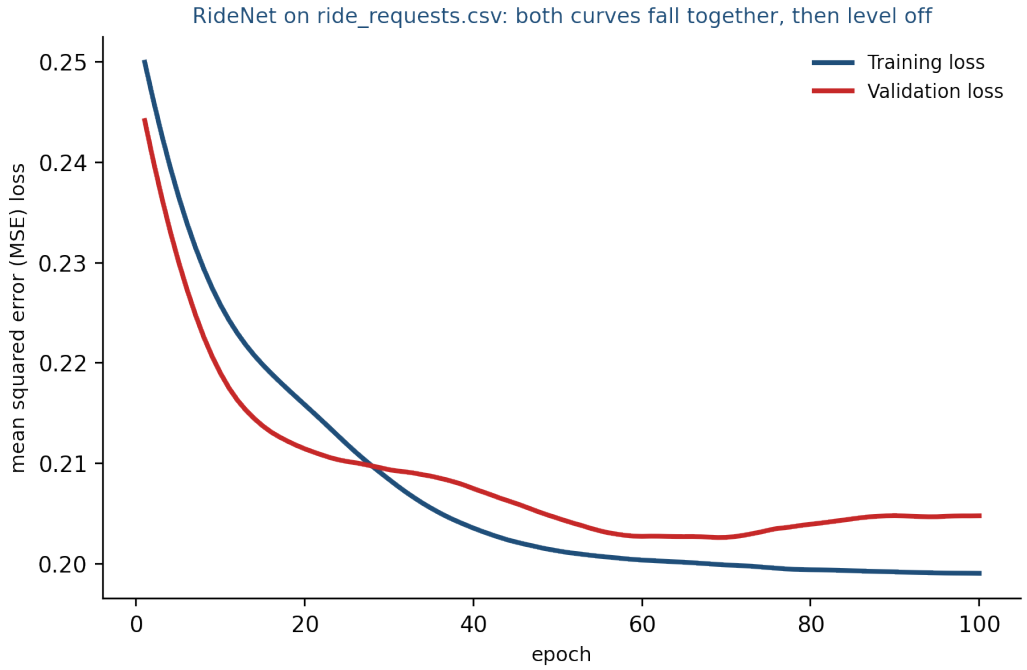
Quick Check: Answer

```
# Gradients from previous batches would accumulate (add up)
# instead of being replaced each time.
#
# The weight updates would end up based on a sum of many
# batches' gradients rather than just the current one, which
# usually makes training unstable or causes it to converge
# to the wrong place.
```

CONCEPT

Reading Your Own Loss Curve

- After each epoch, compute and record the loss on both the training set and the validation set
- A healthy run looks like this: both curves drop together, then flatten out at roughly the same level
- Compare this to Day 19's warning curve, where validation loss started climbing while training loss kept falling
- If you see that gap open up here, the fix is the same as always: simplify the model, add data, or stop training earlier



EXAMPLE

Evaluating Without Training

torch.no_grad() turns off autograd tracking, since no training happens during evaluation.

```
model.eval()
with torch.no_grad():
    probs = model(X_test)
    predicted_classes = (probs > 0.5).float()
    accuracy = (predicted_classes == y_test).float().mean()
    print(f'Test accuracy: {accuracy:.3f}')      # 0.650

from sklearn.metrics import classification_report
print(classification_report(y_test.numpy(), predicted_classes.numpy()))
```

COMPARISON

PyTorch vs. TensorFlow

Both are mature and widely used. Keras is TensorFlow's high-level interface.

	TensorFlow (Keras)	PyTorch
Created by	Google	Meta (originally Facebook AI Research)
Training style	model.fit() runs the loop for you	You write the loop: forward, loss, backward, step
Feel	More built-in structure, less code for standard models	Plain Python, easy to inspect and debug step by step
Strong points	Mature tools for deployment, such as mobile and browser versions	Very popular in research and for modern language models
Same underneath	Tensors, automatic gradients, GPU support	Tensors, automatic gradients, GPU support

RESULTS

Same Network, Three Ways

RideNet (6 inputs, 8 hidden, 1 output, 65 parameters) built three ways and scored on the same 200 test rides, in the order we built them.

Built with	Optimizer and length	Test accuracy	F1
No library (NumPy)	Gradient descent, lr 0.5, 500 epochs	0.67	0.46
TensorFlow (Keras)	Adam, lr 0.01, 100 epochs	0.65	0.42
PyTorch	Adam, lr 0.01, 100 epochs	0.65	0.44

CONCEPT

Why the Numbers Differ

- The network is identical, so the library is not what changes the result
- Random starting weights, the optimizer and the number of epochs do: each run begins from a different point and can end in a slightly different place
- With only 68 quick pickups in the test set, a gap of a few points is within the noise, so no build can be called the winner from one run
- The honest conclusion: on this small table, all three land at or just below Logistic Regression

REFERENCE

Classical Models vs. RideNet

On a small, structured dataset like ride_requests.csv, a tiny neural network rarely beats the classical models.

Model	Typical Fit	When It's the Better Choice
Logistic Regression / Trees	Fast, simple, interpretable	Small, structured/tabular data, most real business problems
RideNet (small NN)	More setup, no real accuracy gain here	Mainly useful as a learning step toward larger networks
Large Neural Network	Needs lots of data and compute	Images, text, audio, and other unstructured, high-volume data

RESULTS

RideNet vs. the Day 18 Models

All five models scored on the same 200 test rides (class 1 = Quick Pickup).

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.70	0.57	0.40	0.47
KNN (K=5)	0.62	0.44	0.38	0.41
Decision Tree (depth 4)	0.68	0.54	0.29	0.38
Random Forest (100 trees)	0.62	0.44	0.46	0.45
RideNet (PyTorch, 100 epochs)	0.65	0.48	0.41	0.44

CONCEPT

What the Comparison Tells Us

- RideNet (PyTorch) lands in the middle: its F1 (0.44) is close to Random Forest (0.45) and trails Logistic Regression (0.47)
- Its accuracy (0.65) is below Logistic Regression (0.70), so the extra complexity did not clearly help on this data
- All three builds score within a few points of each other (accuracy 0.65 to 0.67, F1 0.42 to 0.46), which is within the noise on 200 test rides
- The ride table is small and structured, exactly where classical models are hard to beat
- Neural networks pay off with large amounts of unstructured data such as text, images and audio, which is where this course heads next

CONCEPT

A Few Habits Worth Keeping

- Always scale your inputs, unscaled features can make training unstable or painfully slow
- model.train() and model.eval() switch certain layers' behavior between training and evaluation, call them before each phase
- torch.save(model.state_dict(), 'ridenet.pt') saves a trained model so you don't have to retrain it every time
- Everything here, tensors, the training loop, loss curves, scales up directly to the much larger networks behind modern AI

PRACTICE

Practice

Train RideNet from today for 100 epochs on ride_requests.csv, recording training and validation loss each epoch. Plot both curves. Then compare RideNet's test results to the Logistic Regression, KNN, Decision Tree and Random Forest results from Day 18, which used the same 200 test rides. Which model would you actually ship in the app, and why?

Key Takeaways & Week 4 Wrap-Up

- A PyTorch tensor behaves like a NumPy array, with automatic gradients and optional GPU speed added on top
- Every training loop repeats the same four steps: forward pass, compute loss, backward pass, optimizer step
- The same network can be built with no library, in TensorFlow (Keras), or in PyTorch: frameworks save you the calculus and the bookkeeping, not the thinking
- loss.backward() performs Day 19's backpropagation for you automatically, no calculus required
- This week you built one running project three ways: a fare estimator, a quick-pickup classifier, and a trained neural network, on the same ride data
- Coming up next: moving from structured, tabular data toward the text-based models at the heart of this course